

Αναπαραγωγή και Ανάλυση Αλγορίθμων Randomized Gossip

Αναφορά Προσομοίωσης

Αλυγιζάκης Σπυρίδων

AM: 2021030026

Ιωάννης Φαρμάκης

AM: 2022030002

Σχολή Ηλεκτρολόγων Μηχανικών & Μηχανικών Υπολογιστών

15 Ιουνίου 2026

Περίληψη

Στην εργασία αυτή δοκιμάσαμε να υλοποιήσουμε τους αλγόριθμους Randomized Gossip που περιγράφονται στο paper των S. Boyd et al. (2006). Ο βασικός σκοπός ήταν να δούμε στην πράξη πώς μπορούμε να μειώσουμε το χρόνο που χρειάζεται ένα δίκτυο αισθητήρων για να υπολογίσει κατανεμημένα τον μέσο όρο των μετρήσεών του. Στήσαμε τον κώδικα σε Python, φτιάξαμε τυχαίους γεωμετρικούς γράφους και λύσαμε το πρόβλημα βελτιστοποίησης (SDP) με τη βιβλιοθήκη CVXPY. Τα πειράματά μας επιβεβαίωσαν ότι αν ρυθμίσουμε σωστά τις πιθανότητες επιλογής γείτονα, το δίκτυο συγκλίνει αισθητά πιο γρήγορα σε σχέση με την απλή τυχαία προσέγγιση.

1 Αντικείμενο & Το Μοντέλο

Στα δίκτυα αισθητήρων (sensor networks) συνήθως δεν έχουμε κάποιο κεντρικό server να μαζεύει όλα τα δεδομένα, οπότε δουλεύουμε με αποκεντρωμένους αλγόριθμους. Ένα κλασικό πρόβλημα είναι πώς θα βρουν όλοι οι κόμβοι τον μέσο όρο από τις μετρήσεις τους, και εκεί έρχονται οι αλγόριθμοι gossip.

Η λογική είναι απλή: σε κάθε βήμα, ένας κόμβος διαλέγει στην τύχη έναν γείτονα, μιλάνε, και κρατάνε και οι δύο τον μέσο όρο των τιμών τους. Το ζήτημα είναι να φτάσει όλο το δίκτυο στη σωστή τελική τιμή όσο πιο γρήγορα γίνεται. Στη βασική μορφή του αλγορίθμου, η επιλογή του γείτονα γίνεται με ομοιόμορφη κατανομή (στην τύχη). Στο paper όμως [1] φαίνεται πως αν πειράξουμε λίγο τις πιθανότητες με τις οποίες διαλέγεται η κάθε ακμή, κερδίζουμε πάρα πολύ σε χρόνο σύγκλισης.

Μαθηματικά, έχουμε έναν γράφο $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. Κάθε κόμβος ξεκινάει με μια τιμή $x_i(0)$. Όταν διαλέγεται η ακμή $e = (i, j)$ με πιθανότητα P_e , οι τιμές αλλάζουν βάσει της σχέσης:

$$x(t+1) = W(t)x(t) \quad (1)$$

όπου $W(t) = I - \frac{1}{2}L_e$ και L_e είναι ο τοπικός Laplacian πίνακας της ακμής e .
Ο αναμενόμενος πίνακας βαρών είναι:

$$\bar{W} = \mathbb{E}[W(t)] = I - \frac{1}{2} \sum_{e \in \mathcal{E}} P_e L_e \quad (2)$$

Το πόσο γρήγορα συγκλίνει το σύστημα εξαρτάται αποκλειστικά από τη δεύτερη μεγαλύτερη ιδιοτιμή του $\bar{W}$, δηλαδή την $\lambda_2(\bar{W})$. Οπότε το ζητούμενο της βελτιστοποίησης (το SDP πρόβλημα) είναι να ελαχιστοποιήσουμε αυτή την ιδιοτιμή:

$$\begin{aligned} & \text{minimize} \quad \lambda_{\max} \left(I - \frac{1}{2} \sum_{e \in \mathcal{E}} P_e L_e - J \right) \\ & \text{subject to} \quad \sum_{e \in \mathcal{E}} P_e = 1 \\ & \quad \quad \quad P_e \geq 0, \quad \forall e \in \mathcal{E} \end{aligned}$$

όπου $J = \frac{1}{n} \mathbf{1}\mathbf{1}^T$.

2 Υλοποίηση στην Python

Όλος ο κώδικας γράφτηκε σε VS Code με Python.

2.1 Στήσιμο Γράφου και Solver

Φτιάξαμε Random Geometric Graphs βάζοντας κόμβους τυχαία σε ένα τετράγωνο και ενώνοντάς τους αν απέχουν κάτω από $r = 0.45$. Βάλαμε μια while λούπα για να είμαστε σίγουροι ότι ο γράφος που παράγεται είναι πάντα πλήρως συνεκτικός (διαφορετικά ο gossip δεν δουλεύει καν).

Για να βρούμε το διάνυσμα των πιθανοτήτων P_e , δώσαμε το SDP στη βιβλιοθήκη CVXPY χρησιμοποιώντας τον SCS solver. Αρχικά είχαμε ένα θεματάκι κατά την εκτέλεση, γιατί λόγω numerical errors κάποιες πιθανότητες έβγαιναν ελαφρώς αρνητικές. Για να μην σκάσει ο κώδικας πιο κάτω, αναγκαστήκαμε να προσθέσουμε την `np.clip` στο $[0, 1]$ και να ξανακανονικοποιήσουμε τα βάρη ώστε το άθροισμα να είναι ακριβώς 1.

2.2 Προσομοίωση

Δοκιμάσαμε δίκτυα με $N \in \{12, 20, 30, 40, 50\}$ κόμβους. Οι αρχικές τιμές μπήκαν τυχαία από 0 έως 100. Συγκρίναμε την απλή μέθοδο (**Unoptimized Gossip**), όπου ο κώδικας διάλεγε τυχαία ακμή με `np.random.choice`, και τη βελτιστοποιημένη μέθοδο (**Optimized Gossip**) όπου περνούσαμε στο `choice` και το διάνυσμα των πιθανοτήτων P_e που πήραμε από τον solver. Θέσαμε όριο τα 1000 iterations και αν το Μέσο Τετραγωνικό Σφάλμα (MSE) έπεφτε κάτω από 10^{-6} το κάναμε break.

3 Αποτελέσματα

Ο solver κατάφερε να βρει λύση και για τα 5 σενάρια μεγέθους. Οι καλύτερες τιμές λ_2 βγήκαν ως εξής:

Πίνακας 1: Βέλτιστες τιμές λ_2 ανάλογα με το μέγεθος του δικτύου.

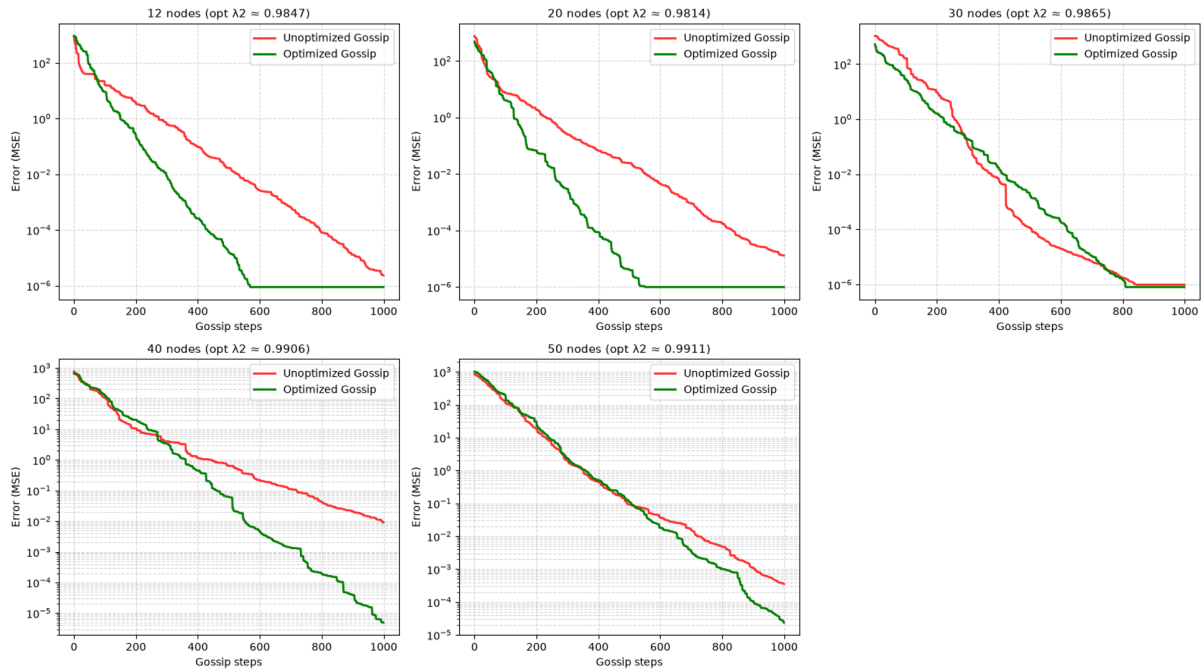
Πλήθος Κόμβων (N)	Βέλτιστο $\lambda_2(\bar{W})$
12	0.98469
20	0.98135
30	0.98654
40	0.99060
50	0.99111

Από τα νούμερα φαίνεται ξεκάθαρα ότι όσο πιο πολλοί είναι οι κόμβοι, τόσο το λ_2 πλησιάζει στο 1. Αυτό βγάζει νόημα, αφού τα μεγαλύτερα δίκτυα χρειάζονται λογικά περισσότερο χρόνο για να απλωθεί η πληροφορία.

Στο Σχήμα 1 έχουμε τα αποτελέσματα από το Optimized τρέξιμο, και στο Σχήμα 2 φτιάξαμε τα plots με λογαριθμική κλίμακα για να φανεί καλύτερα το πώς μειώνεται το σφάλμα.

```
Completed experiment for 12 nodes; optimal  $\lambda_2 = 0.98469$ 
Completed experiment for 20 nodes; optimal  $\lambda_2 = 0.98135$ 
Completed experiment for 30 nodes; optimal  $\lambda_2 = 0.98654$ 
Completed experiment for 40 nodes; optimal  $\lambda_2 = 0.99060$ 
Completed experiment for 50 nodes; optimal  $\lambda_2 = 0.99111$ 
```

Σχήμα 1: Αποτελέσματα για Optimized Gossip.



Σχήμα 2: Σύγκριση εξέλιξης του σφάλματος μεταξύ Unoptimized και Optimized Gossip.

Από τις καμπύλες φαίνεται πολύ καλά η διαφορά. Η πράσινη γραμμή του optimized αλγορίθμου πέφτει πολύ πιο απότομα. Ειδικά στους 50 κόμβους, η απλή μέθοδος σέρνεται ακόμα και στα 1000 βήματα, ενώ η βελτιωμένη μέθοδος πιάνει το όριο αρκετά πιο γρήγορα. Τέλος, παρατηρήσαμε ότι στα μικρά δίκτυα (στους 12 και 20 κόμβους) η βελτιωμένη

μέθοδος έχει τη μεγαλύτερη διαφορά από την απλή, ενώ καθώς οι κόμβοι αυξάνονται αυτή η "ψαλίδα" δείχνει να κλείνει μερικώς.

4 Συμπεράσματα

Βλέποντας την υλοποίηση να τρέχει, καταλαβαίνουμε ότι όντως αξίζει να βελτιστοποιούμε τις πιθανότητες. Ουσιαστικά παίρνεις το θεωρητικό "ταβάνι" του πόσο γρήγορα μπορεί να τρέξει ο gossip στο συγκεκριμένο δίκτυο.

Υπάρχει όμως ένα βασικό πρακτικό θέμα, το οποίο άλλωστε παραδέχεται και το ίδιο το paper: Για να λύσεις το πρόβλημα του CVXPY και να βρεις τα βάρη, πρέπει να ξέρεις από πριν όλη την τοπολογία του δικτύου, και η λύση βγαίνει κεντρικά. Αν μιλάμε για πραγματικά ασύρματα δίκτυα που οι συνδέσεις μπορεί να πέφτουν κάθε τρεις και λίγο, είναι λίγο ανέφικτο να τρέχεις συνέχεια έναν SDP solver από την αρχή. Άρα η συγκεκριμένη μέθοδος μάς φάνηκε χρήσιμη κυρίως για δίκτυα που είναι σχετικά στατικά και μπορούμε να υπολογίσουμε τα βάρη μία φορά και να τα κρατήσουμε.

References

- [1] S. Boyd, A. Ghosh, B. Prabhakar, and D. Shah, "Randomized gossip algorithms," *IEEE Transactions on Information Theory*, vol. 52, no. 6, pp. 2508–2530, June 2006.